

深度学习作业 3

221900180 田永铭 tianym2022@smail.nju.edu.cn

2024 年 12 月 4 日

一、视频理解

Problem 1 分别用简短的语言描述一下单帧 CNN 模型, **Early Fusion**, **Late Fusion**, **3D CNN** 的做法以及区别。

Solution: 做法:

单帧 CNN 模型: 训练普通的 2D CNN 独立对视频单一图象帧进行分类。

Early Fusion: 第一个 2D 卷积处理所有时间信息, 网络的其他部分是常规的 2D 卷积。

Late Fusion: 将每帧的 CNN 特征并联 (或者池化) 并送入 MLP。

3D CNN: 扩展传统的 2D 卷积神经网络, 加入时序维度 T , 使用三维卷积核来处理时序数据或视频帧, 能够捕捉时空信息的变化。

区别:

单帧 CNN 处理的是静态图像, 是最一般, 最难捕捉动态信息的方法。而 **3D CNN** 由于用了三维卷积, 是最适合能够处理时序或视频数据, 能捕捉动态信息的。

其它两种方法介于前两者之间, 都是用“融合”的思想, 但是 **Early Fusion** 是将不同模态的原始数据融合在一起输入网络, 而 **Late Fusion** 则是在每个模态单独处理后再进行融合。

Problem 2 如何使用 RNN (如 LSTM) 建模视频中的长时间依赖? 相比于 3D CNN, 这种方法有何特点?

Solution: 使用 LSTM 进行视频长时间依赖建模:

通常将视频切分成连续的帧序列, 每一帧通过卷积神经网络 (如 ResNet) 提取特征, 之后将这些特征作为 LSTM 网络的输入, 由 LSTM 处理局部特征。基本流程见图 1。

而具体用 LSTM 建模的时候，会考虑更多问题，比如有时为了节省显存，不训练 CNN 预训练并将其用作特征提取器；又如使用多层 RNN 来综合局部时间结构和全局时间结构等等。

■ 建模长时间时序结构

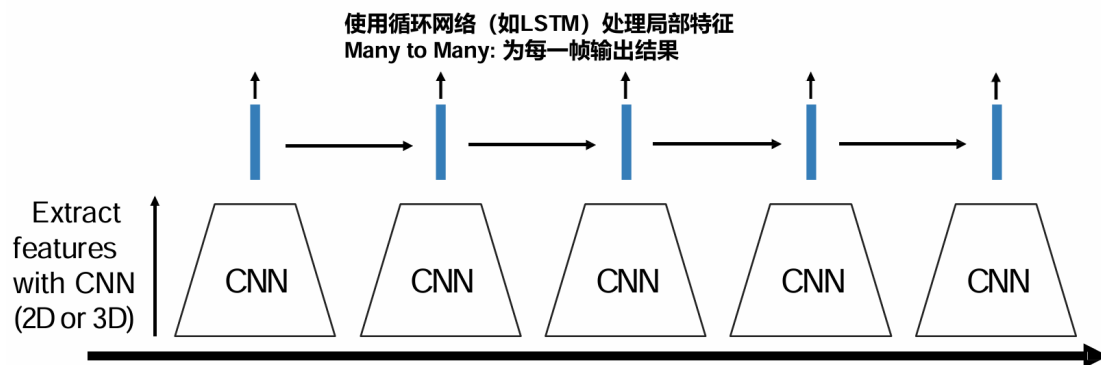


图 1: LSTM 建模长时间序列

相比 3D CNN 的特点:

- **建模方式:** LSTM 逐帧处理视频中的时序信息，通过内部的记忆单元来捕捉长时间的依赖关系，主要解决的是时序信息，空间信息还是由原本的 CNN 来完成。而 3D CNN 通过三维卷积直接对视频的时空数据一起处理，能够同时在空间和时间维度上进行特征提取。
- **捕捉时间依赖的能力:** LSTM 由于由记忆单元等等，相比于 3D CNN 更擅长建模更长时间依赖，能够处理视频中跨多个时间步的长期信息（如情节变化、长时间动作）。
- **灵活度:** LSTM 可处理变长输入（任意帧数视频），而 3D CNN 通常需要固定输入长度（帧数固定）。
- **使用场景:** LSTM 相比 3D CNN 更适合处理长序列。
- **计算效率:** LSTM 在处理长时间序列时计算成本较高，尤其是在长视频中，训练和推理可能变得非常耗时（要计算很多“门”）。而 3D CNN 可以通过多维卷积核并行处理时间帧，通常在处理大规模视频数据时具有更高的计算效率。但是它参数量通常比较大，需要更大的计算资源和内存。

Problem 3 结合课件中 Spatio-Temporal Self-Attention (Nonlocal Block) 部分的介绍，请补充下面的代码。

```
1 import torch
2 import torch.nn as nn
3 class NonLocalBlock(nn.Module):
4     def __init__(self, in_channels):
5         super(NonLocalBlock, self).__init__()
6         self.in_channels = in_channels
7         self.inter_channels = in_channels // 2 # 缩减通道数, 节约计算
8         # 1x1x1 卷积层用于生成 Query, Key, Value
9         self.query_conv = nn.Conv3d(in_channels, self.inter_channels,
10 kernel_size=1)
11         self.key_conv = nn.Conv3d(in_channels, self.inter_channels,
12 kernel_size=1)
13         self.value_conv = nn.Conv3d(in_channels, self.inter_channels,
14 kernel_size=1)
15         # 最终的输出通道映射回原始的 in_channels
16         self.out_conv = nn.Conv3d(self.inter_channels, in_channels,
17 kernel_size=1)
18         # 用于归一化的 softmax
19         self.softmax = nn.Softmax(dim=-1)
20     def forward(self, x):
21         # 输入维度: (N, C, T, H, W)
22         batch_size, C, T, H, W = x.size()
23         # 生成 Query, Key, Value
24         query = self.query_conv(x).view(batch_size, self.inter_channels, -1) # (
25             N, C', T*H*W)
26         key = self.key_conv(x).view(batch_size, self.inter_channels, -1) # (N,
27             C', T*H*W)
28         value = self.value_conv(x).view(batch_size, self.inter_channels, -1) # (N
29             , C', T*H*W)
30         # 请在下面补充完整 Spatio-Temporal Self-Attention 计算过程
31         return out
32 # 测试模块
33 if __name__ == "__main__":
34     # 输入示例: Batch = 2, Channels = 64, Time = 8, Height = 32, Width = 32
35     x = torch.rand(2, 64, 8, 32, 32)
36     nonlocal_block = NonLocalBlock(in_channels=64)
37     out = nonlocal_block(x)
38     print("输入维度:", x.shape)
39     print("输出维度:", out.shape)
```

Solution: 补充部分的代码如下，主要采用的操作是变换形状和矩阵相乘。

```
1 # 计算  $QK^T$ 
2 attn = torch.bmm(query.permute(0, 2, 1), key) # (N, T*H*W, T*H*W)
3 # 归一化
4 attn = self.softmax(attn) # (N, T*H*W, T*H*W)
5 # 计算加权值
6 out = torch.bmm(attn, value.permute(0, 2, 1)) # (N, T*H*W, C')
7 # 恢复原始维度
8 out = out.view(batch_size, self.inter_channels, T, H, W)
9 # 映射回原始通道数
10 out = self.out_conv(out) # (N, C, T, H, W)
11 out = out + x # 残差连接
12 return out
```

生成结果符合预期，输出如图 2，函数作用前后维度不变。

● 输入维度: `torch.Size([2, 64, 8, 32, 32])`
输出维度: `torch.Size([2, 64, 8, 32, 32])`

图 2: 输入输出维度

二、风格迁移与可视化

Problem 4 在风格迁移中，Gram 矩阵的作用是什么？

Solution: Gram 矩阵是图像的 HW 个向量之间两两的内积所组成的矩阵，反映了特征图之间的全局统计相关性，这种相关性与图像的纹理和风格密切相关，所以用于捕捉图像的“风格”信息。具体做法如下：

1. 风格表示：

图像的风格主要由颜色、纹理、形状等元素的统计特征决定。Gram 矩阵通过计算图像中各个特征图（feature maps）之间的相关性来表示这种风格特征。

设图像经过卷积神经网络（CNN）处理后得到一系列特征图（如在某层的输出），Gram 矩阵就是这些特征图的内积矩阵，用于捕捉特征图之间的空间相关性。

2. 计算方法：

假设某一层的特征图大小为 $C \times H \times W$ ，其中 C 是特征图的数量（通道数）， H 和 W 是特征图的高度和宽度。

对于每个特征图 f_i （在二维图像中，通常是特征图的矩阵），Gram 矩阵 G 的计算公式为：

$$G_{ij} = \sum_{h=1}^H \sum_{w=1}^W f_i(h, w) f_j(h, w)$$

即，Gram 矩阵的每一个元素表示特征图 i 和特征图 j 的相关性。

3. 风格迁移：

在风格迁移的过程中，我们希望保持目标图像的内容结构，同时使其风格接近风格图像。因此，风格损失（style loss）是通过计算生成图像和风格图像的 Gram 矩阵之间的差异来度量风格相似性的。

损失函数的一部分通常是：

$$\text{Style Loss} = \sum_l \lambda_l \left(\|G_l^{\text{target}} - G_l^{\text{style}}\|^2 \right)$$

其中， G_l^{target} 和 G_l^{style} 分别是目标图像和风格图像在某层的 Gram 矩阵， l 表示不同层的贡献， λ_l 是该层的权重。

通过这种方式，Gram 矩阵使得风格迁移不仅关注图像的局部细节，还能够捕捉图像全局的风格特征。

Problem 5 风格迁移和快速风格迁移的区别？

Solution: 思想上，从风格迁移到快速风格迁移，有点类似“不再处理单一像素，而是从特征出发”。具体来说，风格迁移需要通过 VGG 进行多次向前/向后传播，速度很慢。快速风格迁移使用前馈网络进行风格迁移，使用预训练的 CNN 计算与以前相同的损失，训练后使用单次前向传播对图像进行风格迁移，速度快。

通过表1可以直观看出区别：

特性	风格迁移（Style Transfer）	快速风格迁移（Fast Style Transfer）
方法	基于优化过程，通过迭代更新生成图像	使用训练好的神经网络进行前馈计算
计算时间	计算时间较长，通常需要几百到几千次迭代	生成速度快，通常可以实时生成图像
生成速度	慢	快（接近实时）
图像质量	较高，细节丰富，风格表现较好	可能有所牺牲，但仍能呈现较好的风格
实现方式	需要每次进行优化和计算损失	通过训练一个生成网络，一次训练后即可使用
计算开销	高，需进行多次前向传播和反向传播	低，一次前向传播即可生成图像

表 1: 风格迁移与快速风格迁移的比较

Problem 6 显著性图是怎么计算的（Saliency Map），它的用途是什么？

Solution: 计算方法: 通过梯度反向传播找到重要的像素。首先进行前向传播, 然后计算每个类预测值相对于图像像素的梯度, 在 RGB 通道上取绝对值的最大值, 具体计算方法如下:

- **Grad-CAM (梯度加权类激活映射):** 通过计算类别分数对最后卷积层输出的梯度, 生成显著性图:

$$\text{Grad-CAM}(x) = \text{ReLU} \left(\sum_k \alpha_k A^k \right)$$

其中, A^k 为卷积层特征图, α_k 为加权系数。

- **CAM (类激活映射):** 直接加权卷积层特征图的每个通道, 生成显著性图, 适用于全局平均池化后的网络结构。
- **基本梯度显著性图:** 通过类别分数对图像像素的梯度计算显著性图:

$$S(x) = \left| \frac{\partial f(x)}{\partial x} \right|$$

其中, $f(x)$ 是网络的输出。

用途:

- **可以用于无监督分割:** 可以根据显著性划分区域。
- **可以用于发现数据偏差:** 比如发现二哈被分类为狼的原因是因为数据图中有雪。
- **模型可解释性:** 帮助理解模型做出预测的依据。
- **特征可视化:** 可视化模型在不同层次学习的特征。
- **数据增强:** 通过关注显著性区域进行数据增强, 提高模型性能。
- **图像编辑:** 用于图像修复、生成等任务, 聚焦重要区域进行操作。

Problem 7 请补充完整下面显著性可视化的代码。

```
1 import torch
2 import torchvision
3 import torchvision.transforms as T
4 import numpy as np
5 import matplotlib.pyplot as plt
6 import requests
7 from PIL import Image
8
9 def download(url, fname):
```

```
10 response = requests.get(url)
11 with open(fname, "wb") as f:
12     f.write(response.content)
13
14 def preprocess(image, size=224):
15     transform = T.Compose([
16         T.Resize((size, size)),
17         T.ToTensor(),
18         T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
19         T.Lambda(lambda x: x[None]),
20     ])
21     return transform(image)
22
23 def deprocess(image):
24     transform = T.Compose([
25         T.Lambda(lambda x: x[0]),
26         T.Normalize(mean=[0, 0, 0], std=[4.3668, 4.4643, 4.4444]),
27         T.Normalize(mean=[-0.485, -0.456, -0.406], std=[1, 1, 1]),
28         T.ToPILImage(),
29     ])
30     return transform(image)
31
32 def show_img(PIL_IMG):
33     plt.imshow(np.asarray(PIL_IMG))
34
35 if __name__ == '__main__':
36     model = torchvision.models.vgg19(pretrained=True)
37     for param in model.parameters():
38         param.requires_grad = False
39
40     download("https://bking.cdn.bcebos.com/pic/3
41             bf33a87e950352ac65cae81db13ecf2b21192131da3?x-bce-process=image/format,
42             f_auto/quality,Q_70/resize,m_lfit,limit_1,w_536","input.jpg")
43     img = Image.open('input.jpg') # 这里可以替换为自己的图片
44     X = preprocess(img) # X.shape: 1, 3, 224, 224
45     model.eval()
46     X.requires_grad_()
47     output = model(X) # 1, 1000
48     print(output.shape)
49     score_max_index = output.argmax()
50     score_max = output[0, score_max_index]
```

```
50 #####
51 # 需要实现课件上显著性可视化部分，具体的步骤为：
52 # a) 通过反向传播获取梯度
53 # b) 在RGB通道上取梯度绝对值的最大值
54 # 请在下面补充代码：
55 #####
56
57 plt.imshow(saliency[0], cmap=plt.cm.hot)
58 plt.axis('off')
59 plt.savefig("output.png")
```

Solution: 补充部分代码如下：

```
1 # a) 通过反向传播获取梯度
2 score_max.backward() # 对最高得分类别反向传播
3 gradients = X.grad.data # 获取输入的梯度
4
5 # b) 在 RGB 通道上取梯度绝对值的最大值
6 saliency, _ = torch.max(gradients.abs(), dim=1) # 取每个像素点在 RGB 通道的
   最大绝对值
7 saliency = saliency.squeeze().cpu().numpy() # 转为 numpy 格式，去掉 batch 维
```

生成结果如图 3。有点原图奶龙的亚子力。😊

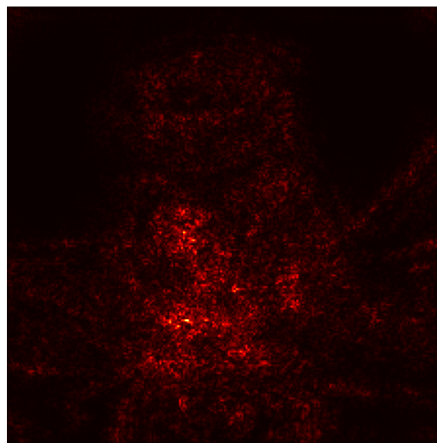


图 3: 显著性图